

Crew — Dossier de projet (Cahier des charges)

Crew — Cahier des charges

Application web responsive pour la planification de services d'équipe, avec solver d'auto-planning, suivi des heures contractuelles et synchronisation calendrier native. Un manager configure son équipe (qui a quel poste, qui travaille combien d'heures), puis génère un planning hebdomadaire en un clic. Les équipiers reçoivent leurs services dans Calendar (iPhone) ou Google Calendar (Android) avec notifications natives 2h avant chaque shift.

Durée : 2 mois — **Projet individuel** — Titre Professionnel DWWM

1. Stack technique

Couche	Technologie
Front	React 18 + Vite + React Router 6
Internationalisation	i18next (FR / EN)
Graphiques	Chart.js + react-chartjs-2
Back	Node.js 22 + Express 4
Base de données	MariaDB 10.11 (driver <code>mysql2</code>)
Authentification	JWT HS256 + bcrypt cost 10
Export calendrier	iCal RFC 5545 + VALARM (notif natif)
Style	CSS pur (custom properties)
Hébergement front	Vercel (build Vite)
Hébergement back	Fly.io (Docker : Node + MariaDB)

2. Concepts métier

Utilisateur

Un utilisateur a un compte personnel (email + mot de passe). Il peut créer une équipe ou rejoindre une équipe existante via un code d'invitation. Un utilisateur peut appartenir à plusieurs équipes (ex : un serveur qui travaille dans deux restaurants).

Équipe

Une équipe est un groupe d'utilisateurs partageant un planning. Chaque membre a un rôle (manager ou équipier). Un manager administrateur peut inviter, valider et retirer des membres.

Membre d'équipe — caractéristiques métier

À l'admission d'un équipier, le manager renseigne :

- **Poste** — zone fonctionnelle d'affectation (cuisine, salle, bar, plonge, administration). Permet au solveur d'affecter les bons profils aux bons services.
- **Shift habituel** — créneau préféré (matin, midi, coupure, soir, nuit). Sert d'indice de score dans la génération automatique.
- **Heures hebdomadaires cibles** — quota contractuel (35h, 24h, etc.). Le solveur répartit les services pour s'en approcher sans dépasser. `NULL` ou `0` = membre exclu du planning automatique (cadres, patron).

Shift

Un shift représente un créneau de travail planifié pour un équipier donné. Champs : équipe, équipier, date, type (matin/midi/.../nuit), poste, note libre, manager créateur. Contrainte d'unicité (`user_id`, `date`, `shift_type`) : un équipier ne peut pas être planifié deux fois sur le même créneau du même jour.

Smart Planner

Algorithme greedy qui propose un planning hebdomadaire complet à partir des caractéristiques de chaque membre. Stratégie de score multicritères :

1. Préférer les membres dont les heures déjà planifiées sont les plus éloignées (en dessous) de leur cible.
2. Bonus si le poste demandé correspond au poste habituel du membre.
3. Bonus si le `shift_type` demandé correspond au `shift_default`.
4. Malus exponentiel à partir du 5^e jour consécutif travaillé.
5. Rejet strict au-delà de 6 jours consécutifs (interdit par le code du travail français).

Le manager voit la proposition dans un modal de preview (heures par membre, slots non couverts, total) avant de l'appliquer. Il peut aussi dupliquer la semaine précédente (`clone-week`) ou effacer toute la semaine en cours (`clear-week`).

3. Rôles et permissions

Action	Manager admin	Manager	Équipier
Inviter / approuver un membre	✓	✗	✗
Modifier rôle / poste / heures	✓	✗	✗
Retirer un membre	✓	✗	✗
Régénérer le code d'invitation	✓	✗	✗
Réinitialiser le mot de passe	✓	✗	✗
Créer / éditer / supprimer un shift	✓	✓	✗
Générer / appliquer un smart planning	✓	✓	✗
Voir tout le planning de l'équipe	✓	✓	✓
Consulter les statistiques équipe	✓	✓	✗
Voir ses propres shifts	✓	✓	✓
S'abonner au flux iCal	✓	✓	✓

4. Modèle de données

Table `users`

```
id          INT AUTO_INCREMENT PRIMARY KEY,  
email       VARCHAR(255) NOT NULL UNIQUE,  
password_hash VARCHAR(255) NOT NULL,  
first_name  VARCHAR(80) NOT NULL,  
last_name   VARCHAR(80) NOT NULL,  
calendar_token CHAR(48) NOT NULL UNIQUE,  
created_at  DATETIME DEFAULT CURRENT_TIMESTAMP
```

Table `families`

```
id          INT AUTO_INCREMENT PRIMARY KEY,  
name        VARCHAR(120) NOT NULL,  
invite_code CHAR(14) NOT NULL UNIQUE,    -- FAM-XXXX-XXXX  
created_by  INT NOT NULL,  
created_at  DATETIME DEFAULT CURRENT_TIMESTAMP,  
FOREIGN KEY (created_by) REFERENCES users(id)
```

Table `family_members`

```
family_id   INT NOT NULL,  
user_id     INT NOT NULL,  
role        ENUM('parent','child') DEFAULT 'child',
```

```

is_admin          BOOLEAN DEFAULT FALSE,
status            ENUM('active','pending') DEFAULT 'pending',
joined_at         DATETIME DEFAULT CURRENT_TIMESTAMP,
poste             ENUM('cuisine','salle','bar','plonge','administration') NULL,
shift_default     ENUM('matin','midi','coupure','soir','nuit') NULL,
weekly_hours_target INT NULL,
PRIMARY KEY (family_id, user_id),
FOREIGN KEY (family_id) REFERENCES families(id) ON DELETE CASCADE,
FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE

```

Table shifts

```

id                INT AUTO_INCREMENT PRIMARY KEY,
family_id         INT NOT NULL,
user_id           INT NOT NULL,
date              DATE NOT NULL,
shift_type        ENUM('matin','midi','coupure','soir','nuit') NOT NULL,
start_time        TIME NULL,
end_time          TIME NULL,
poste             ENUM('cuisine','salle','bar','plonge','administration') NOT
NULL,
note              TEXT NULL,
created_by        INT NULL,
created_at        DATETIME DEFAULT CURRENT_TIMESTAMP,
updated_at        DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
UNIQUE KEY ux_shift_user_day_type (user_id, date, shift_type),
FOREIGN KEY (family_id) REFERENCES families(id) ON DELETE CASCADE,
FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
FOREIGN KEY (created_by) REFERENCES users(id) ON DELETE SET NULL

```

5. API REST

Toutes les routes sont préfixées par `/api`. Sauf `/auth/*` et `/calendar/*`, toutes exigent un header `Authorization: Bearer <jwt>`.

Méthode	Chemin	Description
POST	<code>/auth/register</code>	Création de compte
POST	<code>/auth/login</code>	Connexion → renvoie un JWT
GET	<code>/users/me</code>	Profil de l'utilisateur connecté
PATCH	<code>/users/me</code>	Modifier prénom / nom
POST	<code>/users/me/password</code>	Changer son mot de passe
GET	<code>/families</code>	Mes équipes
POST	<code>/families</code>	Créer une équipe

Méthode	Chemin	Description
GET	/families/:id	Détail d'une équipe + membres
PATCH	/families/:id	Renommer
POST	/families/join	Demander à rejoindre une équipe
POST	/families/:id/approve/:userId	Valider un membre (manager only)
PATCH	/families/:id/members/:userId	Modifier rôle / poste / shift / heures
DELETE	/families/:id/members/:userId	Retirer un membre
POST	/families/:id/members/:userId/reset-password	Réinitialiser le mot de passe d'un membre
POST	/families/:id/regenerate-code	Régénérer le code d'invitation
GET	/shifts?family_id=X&from=Y&to=Z	Lister les shifts
GET	/shifts/upcoming	Mes 10 prochains shifts
POST	/shifts	Créer un shift
PUT	/shifts/:id	Modifier un shift
DELETE	/shifts/:id	Supprimer un shift
GET	/shifts/summary?family_id=X&from=Y&to=Z	Heures par membre + slots non couverts
POST	/shifts/generate-plan	Proposer un planning auto (preview seul)
POST	/shifts/apply-plan	Créer en base les shifts proposés
POST	/shifts/clone-week	Dupliquer une semaine sur la suivante
DELETE	/shifts/clear-week?family_id=X&from=Y&to=Z	Vider tous les shifts d'une semaine
GET	/stats/dashboard	Tableau de bord d'accueil
GET	/stats/charts?family_id=X	Stats équipe (poste, membre, timeline)
GET	/calendar/:token	Flux iCal de tous les shifts de l'utilisateur
GET	/calendar/:token/team/:familyId.ics	Flux iCal d'une équipe spécifique

6. Fonctionnalités fonctionnelles

Compte

- Inscription, connexion, déconnexion, JWT 7 jours
- Modification prénom / nom, changement de mot de passe
- Token calendrier opaque (48 chars hex) régénéré sur demande

Équipe

- Création, renommage, suppression
- Code d'invitation court (FAM-XXXX-XXXX)
- Modération des nouvelles demandes (valider en manager / équipier / refuser)
- Tableau des membres avec badges visuels (poste, shift, heures)
- Setup wizard 3 étapes à la première configuration d'un équipier
- Réinitialisation de mot de passe par l'admin (génère un temp password)

Planning

- Vue grille (membres × jours) sur 7 jours
- Navigation semaine précédente / suivante / cette semaine
- Création / édition / suppression d'un shift par drag-clic
- Couleur par poste, badge type de shift
- Smart Planner : modal preview avec stats par membre + slots non couverts
- Clone week / clear week en un clic
- Indicateur de slots non couverts en haut de la page

Statistiques (managers)

- Camembert : shifts par poste sur 90 jours
- Barres : shifts par membre sur 30 jours
- Courbe : timeline shifts par jour sur 30 jours

Calendrier natif

- Flux iCal personnel (tous les shifts) + flux par équipe
- Alarme VALARM 2h avant chaque shift
- Couleur X-APPLE-CALENDAR-COLOR (Apple uniquement) par équipe
- QR code dans le profil pour faciliter l'abonnement mobile

7. Sécurité

- Hash bcrypt cost 10 sur les mots de passe
- JWT signé HS256, expire à 7 jours, secret en JWT_SECRET
- CORS restrictif : seules les origines listées dans FRONT_ORIGIN peuvent appeler l'API. La valeur supporte une liste comma-séparée pour les environnements multi-domaines (preview Vercel + prod).

- Rate limiting : 10 tentatives de login / 15 min / IP
 - Validation systématique côté back de toute donnée externe
 - Permissions vérifiées sur CHAQUE endpoint via middleware
 - `trust proxy` à 1 pour bonne détection IP derrière Fly proxy
 - HTTPS forcé en production (Fly proxy + Vercel)
-

8. Internationalisation

- Toute chaîne utilisateur passe par `t('clé')` (i18next)
 - Deux locales : `fr.json` et `en.json` (synchronisées)
 - Détecteur de langue navigateur, bouton de switch dans la navbar
 - Format des dates / heures locale via `Intl.DateTimeFormat`
-

9. Déploiement

Front (Vercel)

- Build : `npm run build` → `dist/`
- Routes SPA : `rewrite /(.*) → /index.html` (côté `vercel.json`)
- Env : `VITE_API_BASE=https://crew-back.fly.dev`

Back (Fly.io)

- App : `crew-back` (région `cdg` — Paris)
- Image Docker : `node:22-bookworm-slim` + MariaDB co-localisée
- Volume persistant `crew_db_data` monté sur `/var/lib/mysql`
- `auto_stop_machines = "off"` + `min_machines_running = 1` → pas de cold start pendant la démo
- Secrets : `DB_PASSWORD`, `JWT_SECRET`, `SEED_ON_BOOT`
- Script `start.sh` : démarre MariaDB → migrations → seed (si demandé) → API

Données de démo

Le seed crée une équipe « Bistrot du Vieux Port » avec 8 membres (patron, manager salle, chef cuisine, serveurs, commis, apprentie, plongeur) et un planning de 14 jours pré-rempli (≈ 120 shifts). Permet de présenter le smart planner immédiatement sans configuration manuelle.

10. Critères de validation DWWM

- **CCP1 — Développer la partie front-end** : React + Vite, composants fonctionnels, hooks, i18n, gestion d'état via Context, responsive.
- **CCP2 — Développer la partie back-end** : Express, mysql2, modèle MVC (controllers/models/services), validation, middleware d'auth.

- **CCP3 — Concevoir et développer une application sécurisée organisée en couches** : bcrypt, JWT, CORS strict, rate limiting, RBAC par middleware sur chaque route, validation aux frontières.

Tests : couverture manuelle documentée en annexe, comptes de démo pour chaque rôle, scénarios de cas nominal et cas d'erreur tracés.

11. Évolutions v2 — solver enrichi (migrations 006-012)

Au-delà du périmètre initial décrit dans les sections 1 à 10, le projet a évolué pour intégrer six dimensions métier supplémentaires demandées en cours de développement. Ces évolutions sont entièrement documentées dans `annexes/JUSTIFICATION_SCIENTIFIQUE.md` (12 références peer-reviewed). Le schéma SQL final est synchronisé dans `annexes/SCHEMA_BDD.md`.

11.1 Profils normalisés (migration 006)

Chaque équipier porte un **niveau** (`junior` / `confirme` / `chef`) auquel l'UI applique une grille de **5 profils nommés** : 🌱 Apprenti (0,15), 🌿 Débutant (0,25), 🌳 Autonome (0,40), ⭐ Pilier (0,50), 🏰 Référent (0,60). Tous les poids sont strictement inférieurs à 1 ; chaque poste vise une couverture de 1,00 (= 100 %).

11.2 Paramètres d'établissement (migration 007)

La table `families` reçoit 8 colonnes de configuration métier : coefficients par niveau (`junior_coef` , `confirme_coef` , `chef_coef`), capacité de référence (`max_couverts`) et idéal de couverture par service et par poste (`midi_cuisine_ideal` , `midi_salle_ideal` , `soir_cuisine_ideal` , `soir_salle_ideal`). Le manager les édite via la page Configuration. Anciennes constantes hardcodées supprimées.

11.3 Override personnel et coverage agrégée (008, fractions)

`family_members.coef_override` (nullable) permet de surcharger le poids d'un équipier au cas par cas. L'API `summary` expose `overallService` (moyenne cuisine + salle par service) et la couverture par poste reste accessible via `coverage`.

11.4 Jours d'ouverture et densité prévue (010)

`families.closed_days_mask` (bitmask 7 bits, défaut = lundi fermé) remplace l'ancienne constante. La modale Smart Planner expose un tableau (date × service) où le manager déclare la densité prévue de chaque service de la semaine (Calme 0,5 / Normal 1,0 / Chargé 1,3 ou valeur libre). Le solver scale l'idéal par cette densité.

11.5 Conformité HCR comme contrainte dure

Le solveur applique en filtre quatre garde-fous légaux non contournables : 48 h hebdomadaires (Code du travail L3121-20), 11 h quotidiennes pour cuisinier (HCR niveau I-III) ou 11h30 pour salle/bar (HCR « autre personnel »), 2 jours de repos hebdomadaire (HCR art. 23), maximum 6 jours consécutifs. Aucune suggestion n'enfreint ces seuils ; les modifications manuelles déclenchent des bandeaux d'alerte sur la page Planning citant le texte légal.

11.6 Polyvalence (multi-skill, migration 011)

`family_members.skills_mask` (bitmask 5 bits sur les valeurs de l'enum POSTES). Le solveur `canFill(member, slotPoste)` lit ce mask en priorité ; le scoring conserve un bonus +3 pour le poste primaire, si bien qu'**un équipier polyvalent n'est mobilisé hors de sa spécialité que si aucun spécialiste primaire n'est éligible**. Théorème de la chaîne courte (Jordan & Graves 1995, *Management Science*) appliqué : 2-3 postes/équipier captent ~95 % des gains d'une flexibilité totale.

11.7 Optimisation économique cost-aware (migration 012)

Trois colonnes `*_rate` sur `families` (taux horaire par niveau, en centimes/€) + `family_members.rate_override`. Valeurs par défaut alignées sur les minima de la **Convention HCR 2026** : Junior 12 €, Confirmé 14 €, Chef 19 €. Le solveur intègre une pénalité de coût dans le score de candidature, calibrée pour rester sous-dominante face au déficit hebdomadaire — le coût discrimine seulement entre candidats à besoins équivalents. La masse salariale prévisionnelle (`laborCostTotal`) s'affiche dans l'en-tête de la page Planning.

11.8 Alertes science-based intégrées

Le solveur produit trois familles d'indicateurs visibles dans l'UI, chacun adossé à une source peer-reviewed ou réglementaire (voir annexe scientifique) :

- **fatigueAlerts** — surcharge soutenue (KC & Terwiesch 2009, *Management Science*) : midi + soir > 120 % ou ≥3 services à 130 %+ sur la semaine.
- **hcrViolations** — non-conformités Convention HCR / Code du travail / Matre et al. 2021 (RR=1,24 sur seuils dépassés).
- **serviceHealth** — score composite 0-100 par (jour, service) avec pastille  Saine /  Tendue /  Risque dans l'en-tête des colonnes du planning.

11.9 Récapitulatif des migrations

#	Sujet	Tables impactées
006	Niveaux Junior / Confirmé / Chef	<code>family_members</code>
007	Paramètres d'équipe (coef, ideal, capacité)	<code>families</code>
008	Coef override personnel	<code>family_members</code>

#	Sujet	Tables impactées
009	Normalisation [0;1] des poids et idéals	families , family_members
010	Jours d'ouverture (closed_days_mask)	families
011	Polyvalence (skills_mask)	family_members
012	Taux horaires + override	families , family_members